Day 18 - Lecture 18.5: Multiple Datasets on Line Plot



🌐 **Language / भाषा: English** | हिंदी / Hinglish | ⬅️ Back to Lecture 18 Module

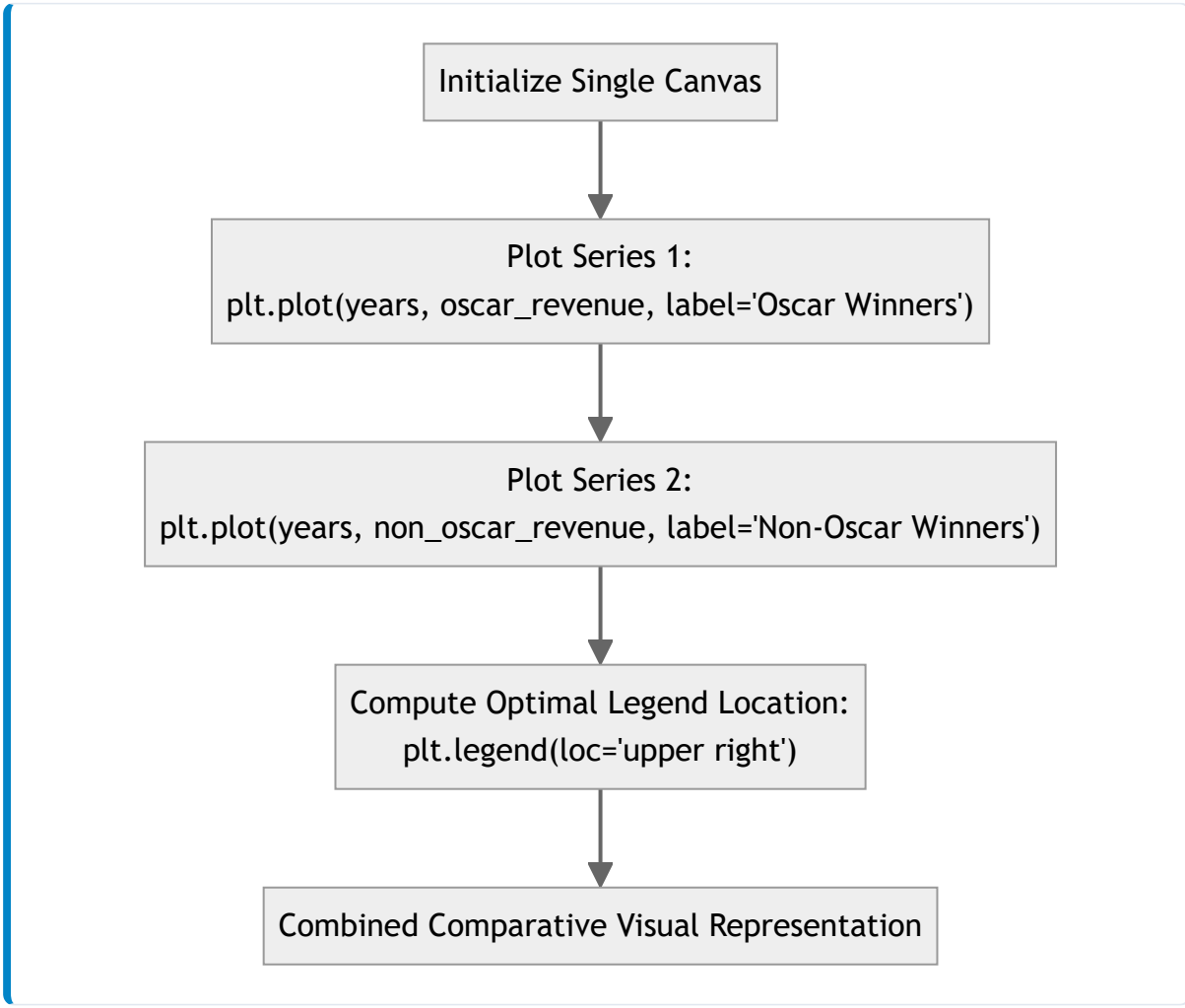
---

1. Overview & Conceptual Foundations

Learn how to overlay multiple independent data series on a single line plot canvas. Use legends, distinct colors, and markers to enable visual comparison between different groups (e.g., Oscar Winners vs Non-Oscar Winners).

---

2. Visual Architecture & Workflow Diagram



---

3. Mathematical & Theoretical Framework

Overlaying K discrete series on a shared domain X:

$$\mathcal{D} = \left\{ \left(X, Y^{(k)}, \ell_k \right) \mid k = 1, 2, \ldots, K \right\}$$

The legend maps visual glyphs (colors c_k, line styles s_k) directly to semantic label strings ℓ_k:

$$\text{Legend} : (c_k, s_k, m_k) \mapsto \ell_k$$

---

4. Structured Implementation & Execution

```
import matplotlib.pyplot as plt

def plot_multiple_time_series(years: list, series_dict: dict, title: str) -> dict:
    """
    Plots multiple comparative series on a single canvas with distinct visual styl
    and a clean legend.

    Parameters:
    -----------
    years : list of int
        Common timeline years.
    series_dict : dict
        Dictionary of format {'Series Name': {'data': list, 'color': str, 'marker'
    title : str
        Plot title.

    Returns:
    --------
    dict
        Computed comparative metrics across all series.
    """
    fig, ax = plt.subplots(figsize=(8, 4.5))
    metrics = {}

    for s_name, s_props in series_dict.items():
        data = s_props['data']
        color = s_props.get('color', None)
        marker = s_props.get('marker', 'o')
        linestyle = s_props.get('linestyle', '-')

        ax.plot(years, data, label=s_name, color=color, marker=marker,
                linestyle=linestyle, linewidth=2.2, markersize=7)

        metrics[s_name] = {
            "total_gross": sum(data),
            "mean_gross": sum(data) / len(data),
            "max_revenue": max(data)
        }

    ax.set_title(title, fontsize=13, fontweight='bold', pad=12)
    ax.set_xlabel("Release Year", fontsize=11)
    ax.set_ylabel("Revenue ($ Millions)", fontsize=11)
    ax.grid(True, linestyle='--', alpha=0.5)
    ax.legend(loc="upper right", framealpha=0.9)

    plt.tight_layout()
    plt.show()
    return metrics
```

```
# Lecture Data: Oscar vs Non-Oscar
years = [2008, 2009, 2010, 2011, 2012]
movie_series = {
    "Oscar Winners": {
        "data": [1005, 170, 427, 133, 232],
        "color": "#1f77b4",
        "marker": "o",
        "linestyle": "-"
    },
    "Non-Oscar Winners": {
        "data": [378, 2788, 829, 185, 275],
        "color": "#d62728",
        "marker": "^",
        "linestyle": "--"
    }
}

comp_results = plot_multiple_time_series(years, movie_series, "Box Office: Oscar v
print("Comparative Financial Metrics:")
for category, val in comp_results.items():
    print(f"  {category}: Total=${val['total_gross']}M, Mean=${val['mean_gross']:.
```

---

5. Summary & Key Takeaways

- **Self-contained Functional Design**: Implementation is encapsulated into modular functions with explicit inputs, structured outputs, and type hints.
- **Visual Accuracy**: Adheres strictly to Matplotlib's layout management, aesthetic guidelines, and production best practices.